

3º Prompt Gemini

Protocolo de Code Review: Prevención de Inyecciones SQL (PHP / MySQL)

1. Objetivo del protocolo

Establecer un procedimiento estándar de revisión de código (*Code Review*) para identificar y corregir vulnerabilidades de inyección SQL (SQLi) en proyectos PHP que interactúen con MySQL. El propósito es garantizar que ningún dato introducido por un usuario pueda alterar la estructura o la lógica de las consultas a la base de datos.

2. Pasos principales durante la revisión

1. **Rastrear la entrada de datos (*Input Tracking*):**
2. Localizar las variables que provengan del exterior, como `$_GET`, `$_POST`, `$_COOKIE`, `$_REQUEST` o cabeceras HTTP.
- 3.
4. **Seguir el flujo de los datos:**
5. Verificar cómo viajan esos datos desde la recepción en el controlador hasta la capa donde se construyen las consultas SQL.
- 6.
7. **Inspeccionar la sentencia SQL:**
8. Analizar la línea de código donde se formula la consulta a la base de datos y comprobar si los datos del usuario se concatenan o se pasan como parámetros aislados.
- 9.
10. **Verificar el uso de la API de base de datos:**
11. Asegurar que se utilicen librerías modernas como **PDO** o **MySQLi** configuradas para utilizar consultas preparadas (*Prepared Statements*).
- 12.

3. Ejemplos de código en PHP: Vulnerable vs. Seguro

Caso 1: Consulta de selección (`SELECT`)

Motivo: Concatenar variables directamente en la cadena SQL permite a un atacante inyectar comandos (por ejemplo, `'OR '1'='1'`). Con consultas preparadas, el motor de base de datos trata la entrada estrictamente como un valor literal.

- **Inseguro (Concatenación de variables):**
-

```
PHP
```

```
// VULNERABLE: Se interpola directamente la variable en la cadena SQL
$email  $_POST 'email'
```

```
$sql "SELECT * FROM usuarios WHERE email = '$email'"
$result $mysqli $sql
```

-
- **Seguro (Consulta preparada con PDO):**
-

PHP

```
// SEGURO: Se usa un marcador de posición (:email) y se vincula el valor aparte
$email $_POST 'email'
$stmt $pdo "SELECT * FROM usuarios WHERE email = :email"
$stmt 'email' $email
$usuario $stmt
```

-

Caso 2: Inserción de datos (INSERT)

Motivo: Los formularios de registro o creación de registros son un vector común de ataque. La parametrización garantiza que caracteres especiales como comillas simples o dobles no rompan la sintaxis SQL.

- **Inseguro (Concatenación con mysqli):**
-

PHP

```
// VULNERABLE: Concatenación directa de múltiples campos
$nombre $_POST 'nombre'
$rol $_POST 'rol'
$sql "INSERT INTO usuarios (nombre, rol) VALUES ('" $nombre "', '" $rol "')"
$mysqli $sql
```

-

- **Seguro (Consulta preparada con mysqli):**
-

PHP

```
// SEGURO: Uso de placeholders '?' y asignación con bind_param
$nombre $_POST 'nombre'
$rol $_POST 'rol'

$stmt $mysqli "INSERT INTO usuarios (nombre, rol) VALUES (?, ?)"
$stmt "ss" $nombre $rol // 'ss' indica dos parámetros de tipo string
$stmt
```

-

4. Errores frecuentes a detectar

- Confiar en `addslashes()` o `mysqli_real_escape_string()` como única defensa:
-

- *Por qué es un error:* Escapar caracteres manualmente es propenso a fallos en ciertos contextos (como en valores numéricos no entrecomillados o según el conjunto de caracteres). No sustituye a las consultas preparadas.
-
- **Omitir el uso de marcadores en campos numéricos:**
-
- *Por qué es un error:* Pensar que `WHERE id = $id` es seguro por ser un número. Si no se ha casteado explícitamente a entero (`((int)$id)`), sigue siendo vulnerable a inyección de texto.
-
- **Concatenar en cláusulas `ORDER BY`, `GROUP BY` o nombres de tablas:**
-
- *Por qué es un error:* Las consultas preparadas no permiten parametrizar nombres de columnas o tablas. Concatenar directamente la entrada del usuario en estas cláusulas abre una brecha de seguridad.
-
- *Solución:* Usar una lista blanca (*allowlist*) de valores permitidos.
-
- **Desactivar las emulaciones de consultas preparadas de forma insegura en PDO:**
-
- *Por qué es un error:* Si `PDO::ATTR_EMULATE_PREPARES` está activado (`true`), PDO puede emular la preparación en el cliente en lugar de usar la nativa de MySQL, lo que en configuraciones concretas de *charset* podría ser vulnerable.
-

5. Checklist final de aprobación

Marca cada casilla para determinar si el Pull Request pasa la revisión de seguridad:

- ☐ **Cero concatenaciones:** Ninguna variable dinámica se concatena con `.` ni se interpola dentro de una cadena SQL.
-
- ☐ **Consultas preparadas aplicadas:** Todas las instrucciones `SELECT`, `INSERT`, `UPDATE` y `DELETE` con datos dinámicos utilizan marcadores de posición (`?` o `:param`).
-
- ☐ **Validación de estructuras dinámicas:** Si se requiere ordenar dinámicamente (`ORDER BY`) o elegir tablas, la entrada del usuario se valida contra una lista estricta de valores permitidos (*allowlist*).
-
- ☐ **Configuración de la conexión segura:**
-

- En PDO: Se ha verificado que las sentencias emuladas estén desactivadas cuando sea necesario
`($pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);`).
-
- Se utilizan el driver PDO o MySQLi (se descartan funciones obsoletas como `mysql_*`).